

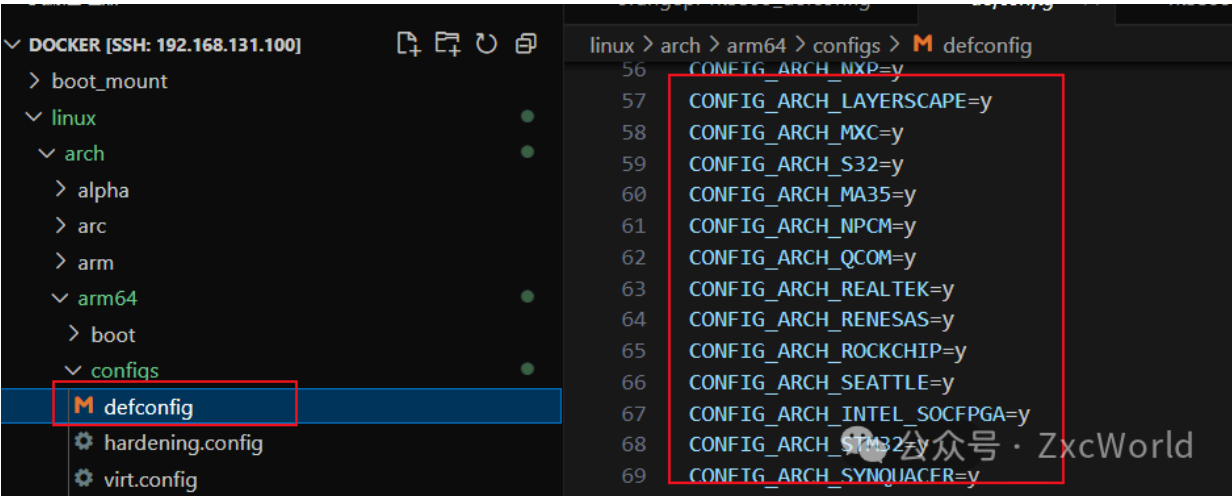
接着上篇的，接下来移植内核和构建根文件。

1.Linux内核

1.1编译内核

进入docker环境，在Linux目录下。

在arm下有很多板子的defconfig，和arm不同，在arm64下的configs中只有一个defconfig，这个里面已经包含了所有的arm64架构的：



但我不需要全部的，所以我拷贝了rk官方库的defconfig，命名为zxc_rk3399_defconfig：

```
1 kernel/arch/arm64/configs/rockchip_linux_defconfig at develop-6.1 · rockchip
```

然后运行：

```
1 make O=rk3399 ARCH=arm64 CROSS_COMPILE=aarch64-none-linux-gnu- zxc_rk3399_defconfig
```

这里指定了一个输出目录`O=`，使用输出目录比较方便查找，而且可以编译多个而不会覆盖掉之前的，推荐使用。

然后编译内核：

```
1 make O=rk3399 ARCH=arm64 CROSS_COMPILE=aarch64-none-linux-gnu-
```

编译完成后获得Image镜像。

1.2烧录内核

像uboot这种启动一般是放在指定地址，而内核和根文件则一般放在文件系统中。

可以使用tftp的方式加载，也可以打包成img烧录，在这里我使用uboot的一个功能指令`ums`（偶然发现），使用ums可以将mmc以块设备的方式暴露给系统,然后可以在虚拟机上，当成正常的存储设备使用，比如使用gpartd进行分区。

ums后用见编译uboot。

先将板子进入uboot，运行uboot命令,这样就变成了一个块设备：

```
1 ums 0 mmc 0
```

然后就可以在虚拟机上看见设备，使用gpartd分一下区就行了。前面uboot实际需要的是固定地址，不需要文件系统，不要分区也可以，最好不要分区，否则会影响后面

的启动脚本，后面两个用于存储内核和根文件，必须是文件系统，我这里将内核的文件系统设置为ext2，rootfs设置为ext4，注意地址偏移(我偏移了20M)，不要将前面3个数据给覆盖了。

挂载后，将内核镜像和dtb拷贝进去就行了，initramfs是一个临时的存于ram的根文件系统，它就是一个根文件，只不过存在于ram中，可以在真正根文件之前做一些初始化，校验等工作，不是必须的，后面会写一下如何通过initramfs运行一个脚本。

2.构建根文件

2.1构建debian

在docker中：

运行脚本(这个过程是官方给出的固定的，所以我直接写成脚本，脚本见第一篇开头),但是有一个很重要，在x86上运行arm64的文件系统，需要`QEMU`,这个在使用chroot切换是很重要。

```
1  ./build-rootfs.sh
```

运行后，生成`rootfs-debian`，这已经是一个根文件了。

可以使用`chroot`切过去，如上所说，必须保证qemu是开启的。

可以使用下面命令查看：

```
1  update-binfmts --display qemu-aarch64
```

没有开启，则需要开启，我的脚本已经执行了这个命令，如果是执行脚本后直接切换说没有问题的，以后再次进入可能会用到:

```
1  update-binfmts --enable qemu-aarch64
```

切换:

```
1  chroot rootfs-debian/
```

可以改下用户名:

```
1  echo "zxc" > ./rootfs-debian/etc/hostname
```

可以改下密码:

```
1  password
```

可以使用apt安装东西:

```
1 apt update
2 apt install ...
```

简而言之就是一个正常Linux根文件。

然后有模块可以使用`make install`安装。

可以将设备树拷贝到`/boot/dts/`中。

将.config拷贝到`/boot`中，名称为config-{内核版本}。

设备树和.config等在基于debian构建initramfs中可能会用到，其实我下面的initramfs是没用的，因为功能过于简单。

拷贝比较麻烦，我构建了一个python脚本,直接运行这个脚本就可以：

```
1 import os
2 import shutil
3 import argparse
4 import subprocess
5
6 def get_kernel_version(kernel_out):
7     try:
8         result = subprocess.run(
9             ['make', 'kernelrelease'],
10            stdout=subprocess.PIPE, #获取标准输入
11            stderr=subprocess.PIPE, #
12            text=True, #文本格式
13            check=True, #错误抛出异常
14            cwd=kernel_out #在kernel_out中运行make
15        )
16        version = result.stdout.strip()
17        print(f"内核版本: {version}")
18        return version
19    except Exception as e:
20        print(f"获取内核版本失败: {e}")
21        return None
22
23
24 def copy_file(src, dst):
25     if os.path.exists(src):
26         os.makedirs(os.path.dirname(dst), exist_ok=True)
27         shutil.copy2(src, dst)
28         print(f"拷贝成功: {src} -> {dst}")
29     else:
30         print(f"Warning: {src} 不存在, 跳过")
31
32 def install_kernel_modules(kernel_out_dir, rootfs_dir, arch="arm64", cross_compile=""):
33     if not os.path.isdir(kernel_out_dir):
34         raise FileNotFoundError(f"内核输出目录没有找到: {kernel_out_dir}")
35     if not os.path.isdir(rootfs_dir):
36         raise FileNotFoundError(f"根文件目录没有找到: {rootfs_dir}")
37
38     cmd = [
39         "make",
40         # f"ARCH={arch}",
41         # f"CROSS_COMPILE={cross_compile}",
42         f"INSTALL_MOD_PATH={os.path.abspath(rootfs_dir)}",
43         "modules_install"
44     ]
45     print(f"Running: {' '.join(cmd)} in {kernel_out_dir}")
46     # 切换到内核输出目录执行
47     try:
48         subprocess.run(cmd, cwd=kernel_out_dir, check=True)
49     except Exception as e:
```

```
50         print("{cmd} 运行失败")
51     print("模块安装成功.")
52 def main():
53     parser = argparse.ArgumentParser(description="拷贝内核数据到debian根文
54     parser.add_argument("--kernel-out", required=True, help="内核编译后输
55     parser.add_argument("--rootfs", required=True, help="debian根文件目录
56     parser.add_argument("--ARCH", default="arm64", help="架构(暂时无用)")
57     parser.add_argument("--CROSS_COMPILE", default="aarch64-none-linux-g
58
59     args = parser.parse_args()
60     version= get_kernel_version(args.kernel_out)
61     if version is None:
62         print("ERROR: 获取内核版本失败, 操作失败")
63         return
64     kernel_out_dir = args.kernel_out
65     rootfs_dir = args.rootfs
66     config_src = os.path.join(kernel_out_dir, ".config")
67     config_dst = os.path.join(rootfs_dir, f"boot/config-{version}")
68     copy_file(config_src, config_dst)
69     install_kernel_modules(kernel_out_dir, rootfs_dir, args.ARCH, args.CRO
70
71
72 if __name__ == "__main__":
73     main()
74
```

运行：

```
1 python3 ./copy_linux_config.py --kernel-out=内核目录 --rootfs=根文件目录
```

最后将根文件上传到mmc：

如内核烧录一样使用uboot的`ums`命令，这个将根文件拷贝到第2个分区,挂载后，使用rsync拷贝过去：

```
1 sudo rsync -aAX ../rootfs-debian/ /mnt/rootfs/
```

2.2构建initramfs

initramfs就是一个根文件，所以说可以使用buildroot之类的构建一个根文件，然后打包成镜像使用，这里直接使用debian的initramfs-tools来构建。

在docker环境中。

切到上面制作debian根文件：

```
1 sudo update-binfmts --enable qemu-aarch64
2 sudo chroot rootfs-debian /bin/bash
```

安装工具：

```
1 apt install initramfs-tools -y
2 apt install u-boot-tools -y
```

添加ls指令,其实本身自带ls，不需要添加，举个例子，可以在这里添加工具，罔。

```
1 cat > /etc/initramfs-tools/hooks/fsck <<'EOF'
2 #!/bin/sh
3 . /usr/share/initramfs-tools/scripts/functions
4 . /usr/share/initramfs-tools/hook-functions
5 copy_exec /bin/ls
```

```
6 EOF
```

修改执行权限：

```
1 chmod +x /etc/initramfs-tools/hooks/fsck
```

创建脚本，这个脚本主要就是这句`ls -l /dev/mmc\*`，显示mmc设备：

```
1 cat > /etc/initramfs-tools/scripts/init-top/checkmmc << 'EOF'
2 #!/bin/sh
3 PREREQ=""
4
5 prereqs() { echo "$PREREQ"; }
6
7 case $1 in
8     prereqs) prereqs; exit 0 ;;
9 esac
10
11 echo "[*] Listing mmc devices..."
12 ls -l /dev/mmc* || echo "No mmc devices found"
13 sleep 3
14 EOF
```

添加执行权限：

```
1 chmod +x /etc/initramfs-tools/scripts/init-top/checkmmc
```

生成镜像，参数`6.12.16-dirty`是内核版本：

```
1 mkinitramfs -c gzip -o /boot/initramfs.gz 6.12.16-dirty
2 mkimage -A arm64 -O linux -T ramdisk -C none -a 0 -e 0 -n initramfs -d /boot/initramfs.gz
```

如果执行了上面的python代码，则会在/boot下生成这个文件，后缀即内核版本，就该示例而言，不要这个是可以，因为我只有一个ls指令，不需要加载其他。

运行后生成如下，将这个也拷贝到内核启动目录中：

3.启动

可以在uboot中手动加载：

```
1 ext2load mmc 0:1 0x40000000 /Image
2 ext2load mmc 0:1 0x10000000 /rk3399-zxc.dtb
3 ext2load mmc 0:1 0x20000000 /initramfs.img
4 setenv bootargs console=ttyS2,1500000n8 root=/dev/mmcblk0p2 rootfstype=ext4
5 booti 0x40000000 0x20000000 0x10000000
```

如果不指定root路径，或者直接不设置bootargs，则会进入到那个initramfs中，这时可以体验到那就是一个简单的根文件。

然后在启动中会打印上面那个ls脚本：

接下来可以构建要给的启动脚本，uboot启动脚本有两种方式`extlinux.conf`和`boot.scr`（这个需要编译，我在v3s中使用的这个，这里不再使用）。

确保uboot开启了这个：`CONFIG\_BOOTMETH\_DISTRO`。

可以在uboot下查看是否支持extlinux启动：

```
1 bootmeth list
```

创建`extlinux.conf`：

```
1 DEFAULT linux
2 TIMEOUT 50
3 LABEL linux
4     MENU LABEL Boot with initramfs
5     KERNEL /Image
6     FDT /rk3399-zxc.dtb
7     INITRD /initramfs.img
8     APPEND console=ttyS2,1500000n8 root=/dev/mmcblk0p2 rootfstype=ext4
9 LABEL no_ramfs
10    MENU LABEL Boot without initramfs
11    KERNEL /Image
12    FDT /rk3399-zxc.dtb
13    APPEND console=ttyS2,1500000n8 root=/dev/mmcblk0p2 rootfstype=ext4
```

然后在启动分区文件系统中创建一个文件夹`/boot/extlinux`：

将`extlinux.conf`拷贝到文件夹中。

然后上电后就会正常启动，进入debian：

进入后就是一个比较完整的Linux，比我那个v3s中使用buildroot构建的好用多了，没办法，谁让v3s只有64M RAM呢，只能说比单片机强点，我感觉称它为大号单片机更加合适。

至此完成。